

DESPLIEGUE Y SEGURIDAD DE UNA PLATAFORMA DE  
TICKETING MULTIENTPRESA EN LA NUBE: **SUPER TICKET**



---

# Índice

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Capítulo 1: INTRODUCCIÓN.....</b>                               | <b>5</b>  |
| 1. 1. Justificación del proyecto.....                              | 5         |
| 1. 2. Objetivo general y específicos .....                         | 7         |
| 1. 2. 1.    Objetivos específicos.....                             | 7         |
| 1. 3. Enfoque de trabajo: desarrollo + despliegue en la nube ..... | 8         |
| <b>Capítulo 2: Estado del arte y tecnologías empleadas.....</b>    | <b>10</b> |
| 2. 1. Revisión de plataformas de ticketing existentes .....        | 10        |
| 2. 2. Justificación de la creación de Super Ticket .....           | 11        |
| 2. 3. Tecnologías seleccionadas .....                              | 12        |
| <b>Capítulo 3: Diseño funcional de Super Ticket.....</b>           | <b>15</b> |
| 3. 1. Descripción general de la plataforma .....                   | 15        |
| 3. 2. Módulos principales del sistema .....                        | 16        |
| 3. 2. 1.    Registro y login.....                                  | 16        |
| 3. 2. 2.    Panel de usuario .....                                 | 16        |
| 3. 2. 3.    Gestión de tickets .....                               | 17        |
| 3. 2. 4.    Rol administrador / gestor .....                       | 18        |
| 3. 3. Soporte multiempresa mediante empresa_id.....                | 18        |
| 3. 4. Posibles elementos visuales (mockups o diagramas UI).....    | 19        |
| <b>Capítulo 4: Arquitectura técnica en la nube.....</b>            | <b>20</b> |
| 4. 1. Diagrama general de arquitectura .....                       | 20        |
| 4. 2. Servicios AWS utilizados.....                                | 20        |
| 4. 2. 1.    Amazon EC2 + ECS (Elastic Container Service) .....     | 20        |
| 4. 2. 2.    Amazon RDS (MySQL) .....                               | 21        |
| 4. 2. 3.    Amazon S3 (Simple Storage Service) .....               | 21        |
| 4. 2. 4.    CloudFront + ACM (HTTPS y CDN).....                    | 22        |
| 4. 2. 5.    IAM, Security Groups y Secrets Manager.....            | 22        |

---

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| 4. 2. 6. CloudWatch (logs y métricas) .....                                   | 23        |
| 4. 3. Terraform: Infraestructura como código .....                            | 23        |
| 4. 4. Justificación del uso del Free Tier.....                                | 24        |
| <b>Capítulo 5: Contenerización y despliegue en AWS .....</b>                  | <b>25</b> |
| 5. 1. Preparación del Dockerfile para Super Ticket.....                       | 25        |
| 5. 2. Definición de la Task en Amazon ECS .....                               | 26        |
| 5. 3. Configuración del ALB con HTTPS .....                                   | 27        |
| 5. 4. Variables de entorno y seguridad .....                                  | 27        |
| 5. 5. Despliegue automatizado con Terraform .....                             | 28        |
| 5. 6. Consideraciones de red, puertos y logs .....                            | 29        |
| 5. 6. 1. Red.....                                                             | 29        |
| 5. 6. 2. Puertos.....                                                         | 30        |
| 5. 6. 3. Logs.....                                                            | 30        |
| <b>Capítulo 6: Adaptación del sistema a multiempresa.....</b>                 | <b>31</b> |
| 6. 1. Creación de la tabla empresas.....                                      | 31        |
| 6. 2. Cambios en el modelo de datos: incorporación de <i>empresa_id</i> ..... | 32        |
| 6. 3. Filtro lógico por empresa en consultas .....                            | 33        |
| 6. 4. Gestión de subdominios o tokens (opcional).....                         | 34        |
| 6. 4. 1. Subdominios personalizados .....                                     | 34        |
| 6. 4. 2. Token de acceso por empresa.....                                     | 34        |
| 6. 5. Pruebas y validación de acceso aislado por empresa.....                 | 35        |
| <b>Capítulo 7: Seguridad y monitorización .....</b>                           | <b>36</b> |
| 7. 1. HTTPS obligatorio con ACM o Let's Encrypt.....                          | 36        |
| 7. 2. Acceso controlado mediante IAM y Security Groups .....                  | 37        |
| 7. 2. 1. IAM (Identity and Access Management).....                            | 37        |
| 7. 2. 2. Security Groups (SGs).....                                           | 37        |
| 7. 3. Registro de logs en CloudWatch.....                                     | 38        |
| 7. 4. Uso de variables secretas cifradas .....                                | 39        |
| <b>Capítulo 8: Resultados obtenidos y pruebas .....</b>                       | <b>40</b> |
| 8. 1. Visualización de la plataforma en funcionamiento .....                  | 40        |
| 8. 2. Rendimiento en la instancia EC2 t2.micro.....                           | 41        |
| 8. 3. Verificación del aislamiento entre empresas.....                        | 41        |

---

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 8. 4. Subida de archivos y almacenamiento en S3.....      | 42        |
| 8. 5. Comprobación del acceso cifrado mediante HTTPS..... | 42        |
| <b>Capítulo 9: Conclusiones y posibles mejoras.....</b>   | <b>44</b> |
| 9. 1. Resumen de logros .....                             | 44        |
| 9. 2. Limitaciones actuales.....                          | 45        |
| 9. 3. Mejoras futuras .....                               | 45        |
| 9. 4. Reflexión final .....                               | 47        |

# Capítulo 1: INTRODUCCIÓN

## 1. 1. JUSTIFICACIÓN DEL PROYECTO

La atención al cliente y la gestión de incidencias son pilares fundamentales en cualquier organización que opere en entornos digitales o presenciales. Con la creciente digitalización de los servicios y el teletrabajo como modalidad extendida, muchas empresas necesitan herramientas accesibles, seguras y eficientes para canalizar las consultas, reclamaciones o incidencias técnicas que reciben de sus usuarios o empleados. En este contexto, las plataformas de gestión de tickets (ticketing) se han consolidado como soluciones clave para automatizar estos procesos, mejorar la trazabilidad de las solicitudes y garantizar tiempos de respuesta aceptables (ITIL Foundation, 2019).

Sin embargo, la mayoría de los sistemas de ticketing actuales presentan diversas limitaciones que dificultan su adopción en entornos pequeños o con múltiples entidades organizativas. Por un lado, las plataformas comerciales como Zendesk, Freshdesk o Jira Service Management suelen implicar costes recurrentes elevados, especialmente cuando se requiere soporte para múltiples empresas o departamentos internos con diferentes políticas de acceso y gestión. Por otro lado, las soluciones de código abierto más populares, como osTicket o Zammad, aunque potentes, presentan una curva de aprendizaje técnica o funcional elevada, además de no estar diseñadas específicamente para una arquitectura multiempresa (o multitenant) desde su núcleo (Bernardino & Martins, 2020).

Ante esta situación, surge la necesidad de diseñar una plataforma de ticketing en la nube, optimizada para pequeñas organizaciones o equipos de trabajo distribuidos, que sea ligera, segura, gratuita y que permita operar de forma diferenciada con varias empresas dentro de una misma instancia. Esta funcionalidad de multitenencia resulta especialmente útil en casos como cooperativas de servicios, grupos empresariales, proveedores TIC con varios clientes o instituciones educativas con departamentos independientes.

El presente proyecto propone el diseño, adaptación y despliegue de una plataforma llamada Super Ticket, desarrollada como una solución sencilla y modular, orientada al registro, seguimiento y respuesta de incidencias o peticiones. Su desarrollo parte de una base funcional previamente disponible como software libre, adaptada y extendida para soportar la gestión de múltiples empresas en un entorno seguro y escalable sobre la nube de Amazon Web Services (AWS). A través de esta iniciativa, se busca demostrar cómo es posible reutilizar y mejorar soluciones de código abierto para crear herramientas modernas y funcionales en condiciones de bajo presupuesto, utilizando exclusivamente servicios gratuitos del AWS Free Tier.

La solución que se presenta ha sido diseñada con una arquitectura distribuida basada en contenedores Docker y orquestación mediante Amazon ECS (Elastic Container Service), junto a una base de datos SQL gestionada (Amazon RDS), almacenamiento seguro en Amazon S3 y cifrado mediante HTTPS (con certificados de Amazon ACM). Esta infraestructura, además de ajustarse a las buenas prácticas de seguridad en la nube (AWS, 2023), permite escalar horizontalmente los componentes del sistema y desacoplar el backend del almacenamiento de datos, lo cual favorece tanto la portabilidad como el mantenimiento del servicio.

En resumen, Super Ticket responde a una necesidad real de muchas organizaciones pequeñas y medianas que requieren gestionar múltiples flujos de incidencias sin incurrir en costes elevados, y que al mismo tiempo desean mantener el control total sobre sus datos. El desarrollo de esta plataforma permite combinar la reutilización de tecnologías existentes con el diseño de una infraestructura moderna y segura en la nube pública, favoreciendo la innovación a través de la adaptación y el despliegue eficiente de recursos abiertos.

## 1. 2. OBJETIVO GENERAL Y ESPECÍFICOS

### Objetivo general

Diseñar, adaptar y desplegar una plataforma de gestión de tickets multiempresa, denominada Super Ticket, utilizando tecnologías de software libre y una arquitectura en la nube basada en servicios gratuitos del nivel Free Tier de AWS.

### 1. 2. 1. OBJETIVOS ESPECÍFICOS

- Analizar y seleccionar una base funcional de código abierto para desarrollar el sistema de tickets, evaluando su potencial de adaptación a una estructura multiempresa.
- Implementar la lógica de multitenencia mediante la inclusión del campo empresa\_id en las principales tablas del modelo de datos (usuarios, tickets, proyectos), permitiendo segmentar el acceso y la gestión por entidad.
- Contenerizar la aplicación web con Docker, asegurando su portabilidad y compatibilidad con entornos de ejecución en la nube.
- Proveer una infraestructura como código con Terraform que cree automáticamente los servicios necesarios en AWS: VPC, instancias EC2, ECS, RDS, S3, CloudFront y ACM.
- Configurar la seguridad y el cifrado del sistema mediante grupos de seguridad, variables de entorno cifradas y autenticación TLS.
- Validar el correcto funcionamiento del sistema desplegado, incluyendo la creación, visualización y resolución de tickets en entornos empresariales diferenciados.
- Evaluar la eficiencia de la infraestructura desplegada, su escalabilidad y sus limitaciones dentro del alcance del Free Tier de AWS.

### 1. 3. ENFOQUE DE TRABAJO: DESARROLLO + DESPLIEGUE EN LA NUBE

El presente proyecto se ha abordado desde un enfoque integral que combina el desarrollo web, la adaptación de software libre, la ingeniería de bases de datos y la arquitectura de sistemas en la nube. El enfoque adoptado ha sido el de un proceso completo de rediseño, adaptación estructural y despliegue en condiciones reales de producción.

El desarrollo de Super Ticket se ha enfocado en resolver una limitación crítica del sistema original: su incapacidad para operar con múltiples entidades organizativas dentro de la misma instancia. Para ello, se ha reestructurado el modelo de datos, añadiendo una tabla *empresas* y el campo *empresa\_id* en las tablas clave como *usuarios*, *tickets* y *proyectos*. Esta modificación, junto a cambios en la lógica de autenticación y filtrado de datos, permite que cada usuario visualice y gestione únicamente los elementos asociados a su empresa, garantizando el aislamiento lógico de los datos.

En paralelo al desarrollo lógico de la aplicación, se ha definido una infraestructura modular en AWS, completamente desplegable mediante Terraform. Esta infraestructura contempla:

- Compute: instancia EC2 tipo t2.micro, alojando una tarea ECS con el contenedor de Super Ticket.
- Contenedores: empaquetado Docker del código, con configuración de puertos, variables de entorno y volúmenes definidos.
- Base de datos: instancia RDS (MySQL) gestionada y asociada a los datos multitenant.
- Almacenamiento de archivos: bucket en S3 para adjuntos y respaldo de información.
- Seguridad: control de acceso mediante IAM, certificados TLS de ACM, y secretos gestionados.
- Red y balanceo: VPC con subredes públicas, Application Load Balancer con HTTPS, y distribución opcional en CloudFront para cacheo y velocidad.



- Monitorización y registros: integración con CloudWatch para el seguimiento de logs, métricas de rendimiento y alertas básicas.

El enfoque de despliegue ha sido progresivo: tras validar el entorno local con Docker Compose, se procedió al empaquetado y envío de la imagen Docker a un repositorio ECR privado, seguido por la creación de una *Task Definition* en ECS y su ejecución dentro de un *Service* asociado al balanceador de carga.

En cuanto a la seguridad, se ha optado por prácticas recomendadas por AWS Well-Architected Framework (AWS, 2022), como el uso de HTTPS en todas las comunicaciones, la gestión de secretos en variables cifradas, y la segmentación de red por grupos de seguridad.

Finalmente, una vez en marcha, se han realizado pruebas funcionales y de carga ligera, confirmando que la plataforma puede operar de forma eficiente en el entorno gratuito de AWS siempre que el uso se mantenga dentro de los límites del Free Tier. De este modo, Super Ticket se consolida como una solución técnica viable y escalable para organizaciones que necesiten gestionar tickets de forma centralizada, diferenciando la experiencia y datos por empresa.

## **Capítulo 2: ESTADO DEL ARTE Y**

### **TECNOLOGÍAS EMPLEADAS**

#### **2. 1. REVISIÓN DE PLATAFORMAS DE TICKETING EXISTENTES**

El ecosistema de plataformas de ticketing ha evolucionado de forma notable en la última década, impulsado por el auge del soporte técnico remoto, la digitalización empresarial y la creciente necesidad de mantener una atención al cliente eficiente y estructurada. Existen múltiples soluciones comerciales y de código abierto que permiten a las organizaciones implementar sistemas de soporte para gestionar incidencias, reclamaciones, consultas o solicitudes de servicio.

Entre las soluciones comerciales, destacan herramientas como Zendesk, Freshdesk, Jira Service Management y Zoho Desk. Estas plataformas ofrecen interfaces modernas, integraciones nativas con múltiples canales (correo electrónico, redes sociales, chat, etc.), y funcionalidades avanzadas como automatizaciones, inteligencia artificial o análisis de rendimiento. Sin embargo, su principal desventaja radica en los costes de suscripción, que aumentan significativamente con el número de usuarios o funcionalidades avanzadas. Además, muchas de estas soluciones no permiten una personalización profunda sin costes adicionales o acceso a capas superiores de pago (Gartner, 2023).

En el ámbito open source, las alternativas más populares incluyen osTicket, Zammad, FreeScout, Faveo Helpdesk y UVdesk. Estas soluciones permiten a los equipos técnicos instalar, configurar y administrar sus propios sistemas de tickets en servidores propios o en la nube, sin pagar licencias. Por ejemplo, osTicket es una plataforma veterana y ampliamente utilizada, desarrollada en PHP y MySQL, que permite gestionar tickets vía correo, formularios web y API. Por su parte, Zammad es una herramienta más moderna, escrita en

Ruby on Rails y AngularJS, que destaca por su interfaz intuitiva y soporte para múltiples canales (Zammad, 2024).

A pesar de sus ventajas, estas herramientas presentan limitaciones importantes para ciertos casos de uso:

- Falta de multitenencia nativa: la mayoría no permiten gestionar múltiples empresas o entornos aislados en una sola instancia. Algunas requieren instancias duplicadas, lo que complica la escalabilidad.
- Sobrecarga técnica en el despliegue: algunas plataformas como Zammad o UVdesk tienen requisitos técnicos elevados (Node.js, Elasticsearch, Redis, configuraciones avanzadas), lo que dificulta su instalación en infraestructuras ligeras o gratuitas.
- Interfaz en inglés o parcialmente traducida: aunque algunas herramientas permiten traducciones, muchas carecen de una localización completa al español por defecto.
- Curva de aprendizaje: herramientas como Jira SM o Faveo, si bien potentes, presentan interfaces complejas que requieren capacitación previa por parte de los usuarios finales.

Estas carencias justifican la necesidad de crear soluciones más ligeras, adaptables, con interfaz en español y que puedan operar de forma eficiente en entornos controlados, como es el caso del nivel gratuito de AWS (Free Tier).

## 2. 2. JUSTIFICACIÓN DE LA CREACIÓN DE SUPER TICKET

Super Ticket surge como respuesta a una necesidad específica: disponer de una solución sencilla, en español y con soporte para múltiples empresas (multiempresa), que pueda ejecutarse de forma económica y segura en la nube, usando recursos gratuitos.

A diferencia de las soluciones existentes, Super Ticket incorpora desde su diseño una arquitectura de datos multitenant, mediante el uso de un campo *empresa\_id* en las tablas clave del sistema. Esto permite que diferentes empresas compartan la misma infraestructura, manteniendo la independencia lógica de sus datos y operaciones. El enfoque está orientado

a organizaciones pequeñas y medianas que requieren un sistema funcional pero no desean incurrir en costes elevados, ni contratar desarrolladores para mantener software complejo.

Además, Super Ticket ha sido diseñado para ser desplegado en AWS utilizando exclusivamente recursos del Free Tier, lo cual permite validar su funcionamiento en un entorno de producción real sin incurrir en costes. Esta característica no es común en soluciones más pesadas, que requieren servidores dedicados o servicios de pago.

A nivel funcional, Super Ticket cubre las necesidades básicas de cualquier sistema de gestión de incidencias: registro de usuarios, creación de tickets, asignación a gestores, seguimiento del estado, categorización por prioridades y notificaciones internas. Su estructura modular permite futuras ampliaciones (p. ej., integración de APIs, dashboards o motores de recomendación), pero su fortaleza principal radica en su sencillez, portabilidad y adaptabilidad.

## **2. 3. TECNOLOGÍAS SELECCIONADAS**

La selección tecnológica de Super Ticket ha estado guiada por tres criterios principales: compatibilidad con AWS Free Tier, sencillez en el despliegue y viabilidad técnica de adaptación.

- Backend: PHP - La aplicación se ha desarrollado sobre PHP, un lenguaje ampliamente soportado por servidores web como Apache o Nginx, y con alta compatibilidad con entornos de contenedores. PHP permite crear aplicaciones funcionales con pocos recursos y facilita la lectura, modificación y extensión del código por parte de desarrolladores de nivel medio o junior. En este caso, se ha partido de una base funcional escrita en PHP procedente del software libre Ticketfly, adaptando sus estructuras de datos, validaciones y vistas para dar soporte a múltiples empresas.
- Base de datos: MySQL (Amazon RDS) - El sistema utiliza una base de datos relacional SQL, gestionada a través de Amazon RDS (Relational Database Service)

en su variante MySQL. Esta elección permite separar el almacenamiento de datos del cómputo, habilitando backups automáticos, recuperación ante fallos, conexión cifrada y escalabilidad futura. Además, RDS ofrece instancias gratuitas bajo el Free Tier (hasta 750 horas/mes con db.t2.micro), lo que permite mantener la base de datos sin coste durante las pruebas o en entornos de baja carga (AWS, 2023).

- **Contenedores y orquestación:** Docker + Amazon ECS (EC2 Launch Type) - La aplicación se empaqueta en una imagen Docker personalizada que contiene PHP, Apache y el código fuente de Super Ticket. Esta imagen se lanza como una tarea en Amazon ECS, utilizando el Launch Type EC2 para aprovechar una instancia gratuita t2.micro del Free Tier. Esto evita el uso de Fargate, que aunque más flexible, no está incluido sin coste. ECS permite definir tareas, servicios, escalabilidad y asociaciones con balanceadores de carga, lo que facilita la operación y mantenimiento del contenedor.
- **Almacenamiento de archivos:** Amazon S3 - Para los archivos adjuntos a los tickets (p. ej., capturas, documentos), se utiliza Amazon S3 como almacenamiento persistente y escalable. Mediante pequeñas adaptaciones del backend en PHP, los archivos se suben al bucket correspondiente en lugar de almacenarse en disco. Esto garantiza durabilidad, accesibilidad y compatibilidad con futuras integraciones (p. ej., enlaces firmados, backups, restricciones por política IAM).
- **Seguridad:** HTTPS, IAM, Secrets Manager - La seguridad se garantiza mediante certificados TLS proporcionados por ACM (AWS Certificate Manager), integrados con un Application Load Balancer (ALB) y, opcionalmente, con CloudFront. Esto permite que todas las comunicaciones entre cliente y servidor estén cifradas. Además, se aplican políticas de acceso granular con IAM, se protegen los recursos por medio de Security Groups y se gestionan credenciales mediante variables de entorno o secretos encriptados.
- **Infraestructura como código:** Terraform - Todo el despliegue de la infraestructura se ha automatizado con Terraform, herramienta de infraestructura como código (IaC)

que permite definir la configuración de AWS en archivos .tf. Esto garantiza reproducibilidad, control de versiones y una administración centralizada de los recursos en la nube. Terraform se ha usado para crear VPC, instancias EC2, ECS Cluster, RDS, S3, ALB, IAM roles, SGs, DNS y certificados, todo de forma declarativa.

## Capítulo 3: DISEÑO FUNCIONAL DE SUPER TICKET

### 3. 1. DESCRIPCIÓN GENERAL DE LA PLATAFORMA

Super Ticket es una plataforma web orientada a la gestión estructurada de tickets de soporte, diseñada para operar en un entorno multiempresa. Su propósito es permitir que distintos usuarios, pertenecientes a organizaciones independientes, puedan registrar incidencias, hacer seguimiento de su resolución y recibir respuestas de parte de los gestores de soporte designados, todo ello dentro de una interfaz unificada, segura y accesible desde cualquier navegador.

La plataforma ha sido concebida con una estructura modular y extensible, desarrollada sobre tecnologías abiertas y fácilmente desplegables en entornos con recursos limitados, como el Free Tier de AWS. A pesar de su sencillez técnica, incorpora funcionalidades clave necesarias en un entorno de atención técnica, tales como clasificación de tickets, asignación de prioridades, notificaciones internas, control de acceso por roles y aislamiento de datos por empresa.

A nivel visual, la interfaz está diseñada de manera responsiva, utilizando estilos CSS básicos (basados en Bootstrap), lo que permite que el sistema sea accesible desde dispositivos móviles y escritorios por igual. Cada módulo del sistema está orientado a un tipo de usuario específico: cliente, gestor o administrador, y su acceso se controla mediante autenticación de sesión tras el inicio de sesión.

## **3. 2. MÓDULOS PRINCIPALES DEL SISTEMA**

### **3. 2. 1. REGISTRO Y LOGIN**

La autenticación es el punto de entrada a la plataforma. Cada usuario accede mediante su correo electrónico y contraseña, los cuales deben estar previamente registrados en la base de datos. Opcionalmente, el sistema puede extenderse para permitir el registro abierto, validación por correo u homologación con directorios externos.

Al momento del inicio de sesión, el sistema recupera el `empresa_id` del usuario, lo cual permite condicionar toda la navegación y los datos accesibles al entorno correspondiente. Este mecanismo es esencial para la separación lógica entre organizaciones.

#### **Campos típicos del login:**

- Email
- Contraseña

#### **Lógica adicional:**

- Verificación activa del usuario
- Asociación automática a una empresa
- Inicio de sesión como cliente o gestor, con rutas diferenciadas

### **3. 2. 2. PANEL DE USUARIO**

Tras el login, los usuarios acceden a un panel general donde pueden:

- Visualizar sus tickets abiertos, cerrados o pendientes
- Crear nuevos tickets
- Ver respuestas de los gestores
- Actualizar su perfil



La información presentada está filtrada por `empresa_id`, lo cual garantiza que cada usuario sólo vea los tickets correspondientes a su propia empresa. En ningún momento es posible acceder a datos de otras organizaciones, ni siquiera por manipulación directa de URLs o IDs.

Este módulo también incluye un resumen visual (indicadores de estado), y enlaces rápidos a creación de tickets o vista detallada.

### **3.2.3. GESTIÓN DE TICKETS**

La creación de tickets se realiza mediante un formulario sencillo con los siguientes campos:

- Asunto del ticket
- Descripción del problema
- Categoría (ej. técnica, administrativa)
- Prioridad (baja, media, alta)
- Posibilidad de adjuntar archivos (subidos a S3)
- Estado inicial: Abierto

Los usuarios pueden crear nuevos tickets, y visualizar el historial de comunicación con los gestores. Cada ticket incluye una sección de comentarios o respuestas internas, que simula una conversación tipo “hilo”, similar a los sistemas de soporte tradicionales.

Los gestores tienen permisos para:

- Cambiar el estado del ticket (pendiente, en curso, cerrado)
- Añadir respuestas visibles para el usuario
- Adjuntar archivos complementarios
- Reasignar la prioridad o categoría
- Ver todos los tickets de su empresa (si tiene permisos de nivel medio)

### 3.2.4. ROL ADMINISTRADOR / GESTOR

- El rol de administrador tiene privilegios ampliados. Desde su panel puede:
- Consultar y modificar todos los tickets de una empresa
- Crear y gestionar usuarios (clientes y gestores)
- Acceder a reportes básicos por categoría o estado
- Gestionar las categorías y prioridades del sistema
- (En algunos casos) gestionar los datos de la empresa asociada

El diseño permite que un único “superadministrador” tenga acceso a todas las empresas, aunque esta funcionalidad está pensada para propósitos de mantenimiento. En el uso normal, cada administrador actúa sobre una única organización (su empresa\_id).

### 3.3. SOPORTE MULTIENTREPRISE MEDIANTE EMPRESA\_ID

La multitenencia se logra a través de una estructura lógica basada en la columna empresa\_id, añadida a las siguientes tablas clave del sistema:

- usuarios (cada usuario pertenece a una empresa)
- tickets (cada ticket pertenece a una empresa, a través del usuario)
- proyectos o categorías (si son específicas de cada empresa)
- respuestas (por relación indirecta al ticket)

Además, se añade una nueva tabla empresas, con campos como:

- id
- nombre
- logo\_url (opcional)
- correo\_contacto
- estado (activa/inactiva)

Durante la autenticación, se recupera el `empresa_id` y se guarda en la sesión. Todos los módulos del sistema filtran sus consultas según este valor, evitando accesos cruzados. Este enfoque, aunque simple, permite que una sola base de datos y una única instancia del sistema sirvan a múltiples clientes de forma completamente aislada.

La lógica de filtrado se implementa mediante condiciones `WHERE empresa_id = ?` en cada consulta SQL. En los controladores y vistas, se emplean validaciones adicionales para evitar que usuarios manipulen identificadores ajenos.

Este diseño es eficiente, escalable a decenas de empresas, y no requiere instancias duplicadas del sistema, lo cual se alinea perfectamente con el uso de una única base de datos gestionada en AWS RDS.

### **3. 4. POSIBLES ELEMENTOS VISUALES (MOCKUPS O DIAGRAMAS UI)**

En la versión completa del informe, se recomienda incluir:

- Figura 3.1: Diagrama de flujo de login y recuperación de `empresa_id`
- Figura 3.2: Vista del panel de usuario (wireframe o captura real)
- Figura 3.3: Ejemplo de formulario de creación de ticket
- Figura 3.4: Diagrama de entidad-relación básico, mostrando la relación entre usuarios, tickets y empresas

Estas figuras ayudarán a visualizar el recorrido funcional de la plataforma y destacar la lógica multiempresa implementada.

## Capítulo 4: ARQUITECTURA TÉCNICA EN LA NUBE

### 4. 1. DIAGRAMA GENERAL DE ARQUITECTURA

Super Ticket se ha desplegado sobre la infraestructura pública de Amazon Web Services (AWS) utilizando únicamente recursos incluidos en el nivel gratuito (Free Tier), lo cual ha permitido validar su funcionamiento en un entorno real sin incurrir en costes económicos.

El sistema se basa en una arquitectura de microservicio monolítico empaquetado en un contenedor Docker, desplegado sobre Amazon ECS (Elastic Container Service) con tipo de lanzamiento EC2, conectado a una base de datos RDS (MySQL), y vinculado a almacenamiento S3 y distribución HTTPS con CloudFront y ACM.

### 4. 2. SERVICIOS AWS UTILIZADOS

#### 4. 2. 1. AMAZON EC2 + ECS (ELASTIC CONTAINER SERVICE)

El componente principal de ejecución es ECS, el sistema de orquestación de contenedores de AWS. Se ha optado por el tipo de lanzamiento EC2 en lugar de Fargate para poder aprovechar las 750 horas gratuitas mensuales de una instancia t2.micro dentro del Free Tier (AWS, 2023). Esta instancia actúa como host para la tarea ECS que ejecuta el contenedor de Super Ticket.

El contenedor incluye:

- PHP 7.4
- Servidor Apache
- Código fuente de Super Ticket

- Configuración de entorno para conexión con RDS y S3

La tarea ECS se define en una Task Definition, donde se especifica el uso de puertos, variables de entorno, políticas de acceso, volumen temporal y recursos de CPU/memoria. El clúster ECS permite gestionar el ciclo de vida de la tarea, escalar horizontalmente si fuera necesario y actualizar el contenedor mediante despliegue continuo.

#### **4.2.2. AMAZON RDS (MySQL)**

La base de datos se gestiona mediante Amazon RDS (Relational Database Service), en su variante MySQL 8.0. Se ha aprovisionado una instancia db.t2.micro, también gratuita durante 750 horas mensuales en el Free Tier, con 20 GB de almacenamiento incluido (AWS, 2023).

RDS aporta ventajas clave:

- Administración automática de parches y backups
- Acceso cifrado y conexión autenticada
- Aislamiento dentro de una subred privada
- Seguridad mediante grupos de seguridad personalizados
- Compatibilidad total con el sistema de autenticación y consultas SQL de Super Ticket

La base de datos contiene todas las tablas relacionales necesarias: usuarios, tickets, empresas, respuestas, etc., incluyendo la lógica multiempresa implementada con empresa\_id.

#### **4.2.3. AMAZON S3 (SIMPLE STORAGE SERVICE)**

Para el almacenamiento de archivos adjuntos en los tickets (como imágenes, PDF o capturas), se utiliza Amazon S3, el servicio de almacenamiento de objetos. Mediante una adaptación del backend PHP, los archivos se suben directamente al bucket S3 usando el SDK oficial de AWS.

Las ventajas de usar S3 son:

- Alta durabilidad (99.999999999% por objeto)
- Escalabilidad automática
- Acceso mediante URLs temporales (firmadas)
- Control granular de acceso con políticas IAM
- Coste nulo en Free Tier (5 GB de almacenamiento y 20.000 GET por mes gratuitos)

Esta integración desacopla completamente el almacenamiento de la infraestructura de cómputo, lo cual es ideal para contenedores efímeros.

#### **4.2.4. CLOUDFRONT + ACM (HTTPS Y CDN)**

Todas las conexiones con Super Ticket se realizan mediante HTTPS gracias al uso de Amazon CloudFront (servicio CDN) combinado con certificados gratuitos de ACM (AWS Certificate Manager).

CloudFront ofrece:

- Distribución global de contenido (bajo latencia)
- Soporte para certificados TLS públicos gestionados por ACM
- Protección contra ataques comunes (DDoS, inyecciones, etc.)
- Integración con WAF (opcional)

El certificado TLS emitido por ACM se asocia al Application Load Balancer (ALB), lo cual permite cifrar las conexiones entre el cliente y el contenedor en ECS de forma transparente y segura.

#### **4.2.5. IAM, SECURITY GROUPS Y SECRETS MANAGER**

La seguridad de la plataforma se ha diseñado siguiendo las buenas prácticas del AWS Well-Architected Framework (AWS, 2022). Se han implementado los siguientes mecanismos:

- IAM (Identity and Access Management): se han creado roles específicos para la tarea ECS, con permisos limitados a S3, logs y RDS.
- Security Groups: cada componente (ECS, RDS, ALB) tiene un grupo que restringe el tráfico entrante y saliente solo a los puertos y orígenes necesarios.
- Secrets cifrados: las credenciales de la base de datos y claves de acceso a S3 se almacenan como variables de entorno cifradas dentro de la definición de tarea ECS (opcionalmente en AWS Secrets Manager).

Este enfoque reduce la superficie de ataque, evita exposición pública de datos sensibles y cumple con los requisitos mínimos de seguridad en la nube.

#### **4. 2. 6. CLOUDWATCH (LOGS Y MÉTRICAS)**

La monitorización de la plataforma se realiza a través de Amazon CloudWatch, que permite centralizar logs y métricas del contenedor ECS, del ALB y de la instancia EC2.

Se han habilitado:

- Logs de acceso y error de Apache
- Logs de eventos de la tarea ECS
- Alarmas básicas de CPU, memoria y errores HTTP

Estos datos permiten diagnosticar fallos, hacer seguimiento de actividad y evaluar el rendimiento sin herramientas externas.

### **4. 3. TERRAFORM: INFRAESTRUCTURA COMO CÓDIGO**

Toda la infraestructura descrita ha sido automatizada mediante Terraform, herramienta de infraestructura como código (IaC). Se han creado módulos reutilizables para:

VPC y subredes públicas

Grupo de seguridad para ECS y RDS

- ECS cluster, definición de tarea y servicio
- ALB + listeners + ACM
- RDS MySQL con parámetros iniciales
- Bucket S3 con política de acceso
- Roles IAM y políticas necesarias
- Variables parametrizadas para despliegue rápido

Este enfoque permite versionar la infraestructura, hacer despliegues reproducibles y modificar componentes sin intervención manual. También facilita la reusabilidad del código para otras instancias de Super Ticket.

#### 4. 4. JUSTIFICACIÓN DEL USO DEL FREE TIER

El Free Tier de AWS ha sido clave en este proyecto, ya que permite crear una arquitectura de producción funcional sin costes durante los primeros 12 meses. Los servicios gratuitos utilizados incluyen:

| Servicio   | Recurso gratuito           | Uso en Super Ticket                 |
|------------|----------------------------|-------------------------------------|
| EC2        | 750 horas/mes t2.micro     | Nodo para ECS y contenedor PHP      |
| RDS        | 750 horas/mes db.t2.micro  | Base de datos MySQL                 |
| S3         | 5 GB + 20.000 GET          | Almacenamiento de archivos adjuntos |
| CloudFront | 1 TB de datos              | CDN y entrega HTTPS                 |
| ACM        | Certificados TLS gratuitos | Seguridad HTTPS                     |
| CloudWatch | Logs básicos               | Registro de actividad y errores     |

Esta combinación permite ejecutar la plataforma con niveles aceptables de carga para 3 empresas sin coste económico, haciendo del proyecto una opción realista para organizaciones pequeñas o en etapa de validación de producto.



## Capítulo 5: CONTENERIZACIÓN Y DESPLIEGUE EN AWS

### 5. 1. PREPARACIÓN DEL DOCKERFILE PARA SUPER TICKET

El primer paso para ejecutar Super Ticket en la nube es su empaquetado en un contenedor Docker, lo cual permite encapsular la aplicación junto con todas sus dependencias (servidor web, intérprete PHP, configuración) en una imagen portable y reproducible.

Dado que Super Ticket está basado en PHP y Apache, se partió de una imagen oficial de Docker:

```
FROM php:7.4-apache

# Instalar extensiones necesarias
RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli

# Copiar archivos de la app
COPY . /var/www/html/

# Activar el módulo de reescritura de Apache
RUN a2enmod rewrite

# Permitir .htaccess
RUN sed -i 's/<Directory \/var\/www\/html\/>\/,/<\/Directory>\/s/AllowOverride None/AllowOverride All/' /etc/apache2/apache2.conf

# Permitir permisos
RUN chown -R www-data:www-data /var/www/html
```

Este **Dockerfile** básico empaqueta el código, configura Apache con **.htaccess** habilitado y permite la conexión a MySQL a través de la extensión **mysqli**. El contenedor queda listo para ser subido a *Amazon Elastic Container Registry (ECR)* o ejecutado directamente en una tarea ECS. Antes de construir la imagen, se eliminan carpetas innecesarias, se ajustan permisos y se agregan archivos de configuración como **.env**, que luego serán sobrescritos por variables de entorno seguras.

## 5. 2. DEFINICIÓN DE LA TASK EN AMAZON ECS

Una vez construido y subido el contenedor, se define una *Task Definition* en Amazon ECS, que especifica:

- Nombre de la tarea
- Imagen del contenedor (URL de ECR)
- Asignación de recursos (0.5 vCPU, 512 MiB RAM)
- Variables de entorno seguras:
  - DB\_HOST, DB\_USER, DB\_PASSWORD, DB\_NAME
  - AWS\_ACCESS\_KEY\_ID, AWS\_SECRET\_ACCESS\_KEY (para acceso a S3 si aplica)
- Montaje de volúmenes temporales (si se desea persistencia efímera)
- Configuración de puertos (exponer el 80)
- Roles de ejecución (*ecsTaskExecutionRole* con políticas mínimas necesarias)

Esta Task Definition se utiliza luego para lanzar un servicio ECS, que garantiza que al menos un contenedor esté corriendo y lo asocia al Application Load Balancer (ALB).

Ejemplo de configuración de variables dentro de la definición de la tarea:

```
"environment": [  
  { "name": "DB_HOST", "value": "super-ticket-db.XXXX.rds.amazonaws.com" },  
  { "name": "DB_USER", "value": "admin" },  
  { "name": "DB_PASSWORD", "value": "*****" },
```

```
{ "name": "DB_NAME", "value": "superticket" }  
]
```

### 5. 3. CONFIGURACIÓN DEL ALB CON HTTPS

Para permitir el acceso cifrado a la aplicación, se implementa un *Application Load Balancer* (ALB) con las siguientes características:

- *Listeners* en el puerto 443 (HTTPS) con certificado TLS gestionado por AWS ACM.
- Redirección automática de puerto 80 a 443.
- *TargetGroup* que apunta a la tarea ECS en el puerto 80.
- *Health check* activo en la ruta `/index.php` para verificar el estado del contenedor.

El certificado TLS se crea con ACM y se asocia al dominio o subdominio público (por ejemplo, `superticket.dominio.com`). Esto asegura cifrado extremo a extremo entre el usuario y el backend.

Toda esta configuración puede automatizarse desde Terraform o ajustarse manualmente desde la consola de AWS.

### 5. 4. VARIABLES DE ENTORNO Y SEGURIDAD

La gestión de variables sensibles se realiza mediante dos mecanismos:

- **Variables de entorno en la *Task Definition***, usadas para credenciales y configuración. Se recomienda almacenar estas variables cifradas o importarlas desde AWS Secrets Manager si se desea un nivel superior de protección.
- **Roles IAM personalizados:**

- El rol de ejecución del contenedor (*ecsTaskExecutionRole*) permite acceso restringido a logs de CloudWatch, buckets de S3 y secretos (si se usa *Secrets Manager*).
- El acceso a RDS está controlado por *Security Groups*, permitiendo tráfico solo desde ECS.

Además, se han establecido políticas restrictivas en los grupos de seguridad:

- ECS: permite tráfico HTTP solo desde el ALB.
- RDS: acepta conexiones únicamente desde el ECS.
- S3: se accede mediante firma de solicitudes con AWS SDK (no se expone públicamente).

Todo el acceso SSH se realiza únicamente a través de la instancia EC2 (si es necesario), usando claves públicas.

## 5. 5. DESPLIEGUE AUTOMATIZADO CON TERRAFORM

Para evitar configuraciones manuales propensas a error, se ha optado por un enfoque de Infraestructura como Código (IaC) utilizando Terraform.

Los módulos definidos permiten:

- Crear la VPC, subredes públicas, y tablas de rutas.
- Desplegar la instancia EC2 y configurar el cluster ECS.
- Definir la Task Definition con variables.
- Crear el servicio ECS y asociarlo al ALB.
- Provisionar RDS con parámetros definidos.
- Crear y configurar el bucket S3.
- Generar el certificado TLS con ACM.
- Crear el Load Balancer y los listeners HTTPS.

### Un fragmento simplificado de Terraform para ECS:

```
resource "aws_ecs_task_definition" "super_ticket" {
  family            = "super-ticket"
  requires_compatibilities = ["EC2"]
  network_mode      = "awsvpc"
  cpu               = "512"
  memory            = "1024"
  execution_role_arn = aws_iam_role.ecs_execution.arn

  container_definitions = jsonencode([
    {
      name       = "super-ticket"
      image      = "<URL_ECR>"
      portMappings = [{ containerPort = 80, hostPort = 80 }]
      environment = [
        { name = "DB_HOST", value = var.db_host },
        ...
      ]
    }
  ])
}
```

Al ejecutar *terraform apply*, toda la infraestructura queda desplegada de forma automática y coherente.

## **5. 6. CONSIDERACIONES DE RED, PUERTOS Y LOGS**

### **5. 6. 1. RED**

Se ha creado una VPC con subredes públicas, donde se ubican:

- El Load Balancer

- La instancia EC2 que actúa como host para ECS

La base de datos RDS se encuentra en una subred privada, sin IP pública, y protegida mediante grupos de seguridad.

### **5.6.2. PUERTOS**

- ALB: escucha en 80 (HTTP) y 443 (HTTPS)
- ECS: expone el puerto 80 del contenedor
- RDS: puerto 3306 abierto solo a ECS

### **5.6.3. LOGS**

Los registros se gestionan con CloudWatch Logs, que recopila:

- Logs de Apache
- Logs de error de PHP
- Logs del servicio ECS
- Logs del sistema EC2 (si se desea)

Esto permite hacer seguimiento de errores, analizar peticiones y depurar problemas de red o permisos.

## Capítulo 6: ADAPTACIÓN DEL SISTEMA A

### MULTIEMPRESA

#### 6. 1. CREACIÓN DE LA TABLA EMPRESAS

Uno de los principales objetivos del proyecto Super Ticket es soportar múltiples empresas (o clientes) dentro de una única instancia de la plataforma. Para lograr esto, el primer paso ha sido la creación de una tabla dedicada a almacenar los metadatos de cada empresa registrada en el sistema.

La tabla *empresas* actúa como entidad principal de referencia para todos los elementos del sistema. Su estructura básica es la siguiente:

```
CREATE TABLE empresas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  logo_url VARCHAR(255),  
  email_contacto VARCHAR(100),  
  estado ENUM('activa', 'inactiva') DEFAULT 'activa',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Esta tabla permite no solo identificar a cada empresa, sino también asociarle configuraciones específicas, como su logotipo o email de contacto. Si se desea escalar el sistema a decenas o cientos de organizaciones, esta estructura facilita la expansión futura.

## 6. 2. CAMBIOS EN EL MODELO DE DATOS: INCORPORACIÓN DE *EMPRESA\_ID*

Una vez definida la entidad *empresas*, se procedió a modificar las principales tablas del sistema para añadir el campo *empresa\_id*, con el objetivo de establecer la relación jerárquica y lógica entre cada elemento del sistema y su empresa correspondiente.

Las tablas modificadas incluyen:

- *usuarios* → cada usuario pertenece a una empresa.
- *tickets* → cada ticket está vinculado al usuario (y, por extensión, a la empresa).
- *proyectos* o *categorías* → si son personalizadas por empresa, también se asocian a *empresa\_id*.

### Ejemplo: tabla *usuarios* modificada

```
ALTER TABLE usuarios ADD COLUMN empresa_id INT NOT NULL;  
  
ALTER TABLE usuarios  
ADD CONSTRAINT fk_usuarios_empresas  
FOREIGN KEY (empresa_id) REFERENCES empresas(id)  
ON DELETE CASCADE;
```

### Ejemplo: tabla *tickets* modificada

```
ALTER TABLE tickets ADD COLUMN empresa_id INT NOT NULL;  
-- En este caso, empresa_id se llena al momento de crear el ticket,  
-- basándose en el empresa_id del usuario autenticado
```

Estas modificaciones permiten asegurar que todas las operaciones (creación, lectura, actualización, eliminación) se realicen siempre en el contexto de la empresa correspondiente.



### 6. 3. FILTRO LÓGICO POR EMPRESA EN CONSULTAS

Una vez añadidos los campos *empresa\_id*, es fundamental garantizar que toda la lógica del sistema respete esta segmentación. Para ello, se implementaron filtros lógicos en las consultas SQL, tanto en los controladores del backend como en las vistas de usuario.

#### Ejemplo de filtro en consultas (PHP + MySQL)

```
// ID de empresa recuperado de la sesión
$empresa_id = $_SESSION['empresa_id'];

$sql = "SELECT * FROM tickets WHERE empresa_id = ? ORDER BY creado DESC";
$stmt = $pdo->prepare($sql);
$stmt->execute([$empresa_id]);
```

#### En la creación de tickets:

```
// Cuando un usuario crea un ticket, su empresa_id se asocia automáticamente
$empresa_id = $_SESSION['empresa_id'];

$sql = "INSERT INTO tickets (titulo, descripcion, empresa_id, usuario_id)
VALUES (?, ?, ?, ?)";
$stmt = $pdo->prepare($sql);
$stmt->execute([$titulo, $descripcion, $empresa_id, $usuario_id]);
```

Este enfoque garantiza que cada empresa solo vea sus propios datos, sin necesidad de instanciar varias bases de datos o clonar la aplicación. La seguridad de acceso queda reforzada por validaciones en la lógica de negocio y la interfaz.

Además, todas las rutas protegidas del sistema cargan primero `empresa_id` desde la sesión activa y lo utilizan como condición global para limitar el acceso.

## 6. 4. GESTIÓN DE SUBDOMINIOS O TOKENS (OPCIONAL)

Aunque el sistema funciona perfectamente con autenticación clásica y filtrado por sesión, se contempla la posibilidad de añadir una capa adicional de diferenciación visual o lógica por medio de subdominios o tokens de empresa.

### 6. 4. 1. SUBDOMINIOS PERSONALIZADOS

Cada empresa podría acceder a su propio espacio mediante una URL tipo:

```
empresa1.superticket.com  
empresa2.superticket.com
```

Esto requiere configurar DNS y reglas en CloudFront o el Application Load Balancer (ALB) que redirijan según el subdominio. En el backend, se puede extraer el subdominio con PHP:

```
$host = $_SERVER['HTTP_HOST'];  
$subdominio = explode('.', $host)[0];
```

Luego, se asocia `empresa_id` con el subdominio correspondiente en la base de datos.

### 6. 4. 2. TOKEN DE ACCESO POR EMPRESA

Como alternativa, se puede implementar una URL con token identificador:

```
superticket.com/login?empresa=empresa1
```

Este token puede usarse para filtrar dinámicamente el contenido y aplicar temas personalizados. Sin embargo, esta técnica es menos robusta que el uso de subdominios, especialmente si se desea una separación visual y operativa más marcada.

## **6. 5. PRUEBAS Y VALIDACIÓN DE ACCESO AISLADO POREMPRESA**

Para verificar la correcta implementación del modelo multiempresa, se realizaron pruebas manuales con usuarios pertenecientes a distintas empresas. Se validaron los siguientes escenarios:

- Un usuario de Empresa A solo puede ver sus propios tickets.
- Los gestores de Empresa B no pueden acceder a los datos de Empresa A.
- Las rutas protegidas cargan únicamente los recursos filtrados por `empresa_id`.
- Los intentos de manipular identificadores externos (por ejemplo, accediendo a `/ticket?id=999`) son bloqueados si el ticket no pertenece a la empresa del usuario.
- Los formularios de creación de tickets rellenan automáticamente el campo `empresa_id` con la empresa activa.

Estas pruebas confirman que la adaptación del sistema ha sido exitosa, permitiendo operar con varias empresas de forma segura, aislada y eficiente dentro de una única instancia de Super Ticket.

## Capítulo 7: SEGURIDAD Y MONITORIZACIÓN

La seguridad es un pilar fundamental en el diseño y despliegue de cualquier sistema expuesto a internet. En Super Ticket, se han aplicado principios de seguridad en todos los niveles de la arquitectura, desde la transmisión de datos hasta el acceso a los recursos internos, utilizando herramientas y servicios proporcionados por AWS y buenas prácticas recomendadas por el marco *AWS Well-Architected* (AWS, 2022).

### 7. 1. HTTPS OBLIGATORIO CON ACM O LET'S ENCRYPT

Para proteger las comunicaciones entre los usuarios y el sistema, se ha establecido el uso obligatorio de *HTTPS (Hypertext Transfer Protocol Secure)*. Este protocolo garantiza la confidencialidad e integridad de los datos en tránsito, impidiendo que terceros intercepten o modifiquen la información intercambiada entre cliente y servidor.

El cifrado *TLS (Transport Layer Security)* ha sido implementado mediante dos alternativas viables:

- **ACM (AWS Certificate Manager):** se generó un certificado SSL gratuito a través de ACM, válido para un dominio personalizado (por ejemplo, `superticket.example.com`). Este certificado se integró con el Application Load Balancer (ALB), que gestiona las conexiones entrantes al contenedor ECS.
- **Let's Encrypt (opcional):** como solución alternativa o para entornos fuera de AWS, se puede instalar Certbot en un contenedor Nginx que actúe como proxy inverso. Esta opción permite renovar automáticamente certificados TLS sin intervención manual.

El ALB está configurado para:

- Redirigir todas las conexiones HTTP (puerto 80) a HTTPS (puerto 443).

- Aceptar únicamente conexiones cifradas.
- Mantener la compatibilidad con navegadores modernos.

Con ello, todas las páginas de la aplicación y sus operaciones internas (como login, subida de archivos, creación de tickets) se realizan a través de un canal cifrado.

## **7.2. ACCESO CONTROLADO MEDIANTE IAM Y SECURITY GROUPS**

La seguridad interna se ha reforzado mediante el uso de mecanismos propios de AWS:

### **7.2.1. IAM (IDENTITY AND ACCESS MANAGEMENT)**

Se han definido roles IAM específicos para las tareas ECS. Estos roles se asocian a permisos mínimos necesarios (principio de mínimo privilegio), incluyendo:

- Lectura/escritura en un bucket S3 determinado.
- Acceso a logs de CloudWatch.
- Recuperación de secretos o parámetros (si se usa AWS Secrets Manager o SSM Parameter Store).

Cada contenedor ECS solo hereda permisos mediante el rol de ejecución de tarea, evitando el uso de credenciales estáticas dentro del contenedor.

### **7.2.2. SECURITY GROUPS (SGs)**

Los Security Groups funcionan como cortafuegos virtuales a nivel de red. Se han aplicado las siguientes reglas:

- ECS Service: solo permite tráfico HTTP desde el ALB (no desde internet).
- ALB: acepta tráfico público en 443 (HTTPS) y redirige desde 80 (HTTP).
- RDS MySQL: solo acepta conexiones entrantes desde la subred del ECS (no expuesto públicamente).

- S3: restringido mediante políticas IAM, no accesible de forma anónima.

Este modelo de red cerrada garantiza que ningún componente del sistema esté expuesto más allá de lo estrictamente necesario, minimizando riesgos de intrusión.

### **7. 3. REGISTRO DE LOGS EN CLOUDWATCH**

Para garantizar la trazabilidad de las acciones y facilitar la monitorización del sistema, se ha integrado Amazon CloudWatch Logs. Desde la definición de la tarea ECS, se ha habilitado el envío automático de logs al servicio de AWS.

Los logs recopilados incluyen:

- Salida estándar del contenedor (STDOUT/STDERR).
- Logs del servidor Apache (accesos y errores).
- Errores de PHP (avisos, excepciones).
- Eventos de despliegue y reinicio de tareas ECS.

Estos registros se agrupan por nombre de servicio y contenedor, lo que permite consultar fácilmente los eventos por periodo de tiempo o nivel de severidad. También es posible definir alarmas personalizadas si se detecta un número elevado de errores HTTP, consumo anormal de recursos, o interrupciones del servicio.

Además, CloudWatch puede integrarse con AWS SNS para enviar notificaciones automáticas por correo electrónico o SMS en caso de incidentes críticos.

## 7. 4. USO DE VARIABLES SECRETAS CIFRADAS

En lugar de almacenar contraseñas o claves API directamente en archivos del proyecto, se ha optado por gestionar las credenciales de forma segura:

- Las variables como DB\_PASSWORD, AWS\_SECRET\_ACCESS\_KEY y tokens de acceso se declaran como variables de entorno cifradas dentro de la definición de tarea ECS.
- Opcionalmente, estas variables pueden moverse a AWS Secrets Manager o SSM Parameter Store, con cifrado automático (KMS) y control de acceso mediante políticas IAM.

Al centralizar y cifrar la configuración sensible:

- Se reduce el riesgo de exposición accidental en el código fuente.
- Se facilita el control de accesos por entorno (dev, test, producción).
- Se garantiza la rotación segura de claves, si fuera necesario.

Este enfoque cumple con las recomendaciones del pilar de seguridad del AWS Well-Architected Framework, que promueve la separación entre código y secretos (AWS, 2022).

En conjunto, las medidas de seguridad y monitorización aplicadas en Super Ticket aseguran que la plataforma pueda operar en un entorno expuesto a internet de forma fiable, cumpliendo estándares básicos de confidencialidad, integridad y disponibilidad.

## **Capítulo 8: RESULTADOS OBTENIDOS Y**

### **PRUEBAS**

Una vez desplegada la plataforma Super Ticket en AWS utilizando los recursos gratuitos del nivel Free Tier, se llevaron a cabo diversas pruebas funcionales y de rendimiento. Estas pruebas permitieron validar tanto el correcto funcionamiento de la plataforma como su comportamiento en escenarios reales de uso multiempresa, asegurando también el cumplimiento de las medidas de seguridad implementadas.

#### **8. 1. VISUALIZACIÓN DE LA PLATAFORMA EN FUNCIONAMIENTO**

Durante la fase de validación, se obtuvieron capturas de pantalla de los distintos módulos de la aplicación ya operativa. En primer lugar, se verificó el correcto funcionamiento del sistema de autenticación, que permite a los usuarios iniciar sesión desde una interfaz sencilla y cifrada. Una vez dentro del sistema, cada usuario puede visualizar su propio panel, donde aparecen únicamente los tickets que ha creado o que pertenecen a su empresa.

También se comprobó el formulario de creación de tickets, que permite incluir título, descripción, y adjuntar archivos. La interfaz del panel de administración, reservada a usuarios gestores, mostró correctamente los tickets activos asociados a la empresa correspondiente. Todo el sistema opera en castellano, facilitando su uso para entornos hispanohablantes. Las capturas evidencian que el sistema cumple con los requisitos funcionales básicos de una plataforma de ticketing moderna y multiempresa.



## 8. 2. RENDIMIENTO EN LA INSTANCIA EC2 T2.MICRO

La infraestructura fue desplegada sobre una instancia EC2 tipo t2.micro, con 1 vCPU y 1 GB de RAM, incluida dentro del Free Tier de AWS. Esta instancia se utilizó como host para ejecutar el servicio ECS que contiene la aplicación.

Las pruebas demostraron que Super Ticket puede funcionar de manera fluida en este entorno, siempre que el número de usuarios concurrentes sea limitado. En situaciones normales, con uno o dos usuarios navegando por la plataforma, el tiempo de carga promedio fue inferior a un segundo, sin experimentar bloqueos ni ralentizaciones. El consumo de CPU se mantuvo en niveles moderados, y la memoria se utilizó eficientemente sin agotar los recursos asignados. Aunque la *t2.micro* no es adecuada para producción intensiva, resultó completamente válida para demostraciones, pruebas funcionales y desarrollo.

## 8. 3. VERIFICACIÓN DEL AISLAMIENTO ENTRE EMPRESAS

Uno de los elementos clave del proyecto fue asegurar que cada empresa registrada tuviera acceso únicamente a sus propios datos. Para ello, se realizaron múltiples pruebas de acceso cruzado entre cuentas de distintas organizaciones. Se comprobó que los usuarios de una empresa no podían ver ni manipular los tickets de otra, ni accediendo directamente por la interfaz ni alterando manualmente las URLs o parámetros internos.

De igual modo, se confirmó que todos los tickets creados se asociaban automáticamente a la empresa correspondiente mediante la variable *empresa\_id*, y que las consultas SQL estaban filtradas por este identificador. El sistema respetó en todo momento la compartimentación entre empresas, cumpliendo con el objetivo de ofrecer una instancia única de Super Ticket capaz de servir simultáneamente a varias organizaciones sin comprometer la seguridad o privacidad de los datos.

## **8. 4. SUBIDA DE ARCHIVOS Y ALMACENAMIENTO EN S3**

Super Ticket permite que los usuarios adjunten archivos a los tickets, ya sea para aportar documentación, imágenes, registros u otro tipo de soporte. Estos archivos no se almacenan en la máquina local, sino que se suben directamente a un bucket de Amazon S3 configurado como privado.

Durante las pruebas, se subieron correctamente archivos PDF, imágenes JPG y capturas PNG, cada uno con un tamaño de hasta 5 MB. La subida fue rápida, y la disponibilidad de los archivos quedó garantizada. Se verificó que ningún archivo fuera accesible públicamente sin autorización, y que los permisos del bucket restringían el acceso solo a la aplicación mediante políticas IAM. Estas pruebas demostraron que el sistema de almacenamiento de archivos es seguro, escalable y se integra correctamente con el flujo de creación y visualización de tickets.

## **8. 5. COMPROBACIÓN DEL ACCESO CIFRADO MEDIANTE HTTPS**

Todas las operaciones en Super Ticket se realizaron desde un dominio protegido por HTTPS. El certificado TLS fue gestionado automáticamente por AWS Certificate Manager (ACM) y asociado al Application Load Balancer que da acceso al servicio ECS.

Durante las pruebas de navegación, se confirmó que todas las páginas del sistema cargaban con cifrado activo, y que los navegadores reconocían el certificado como válido, mostrando el candado de seguridad en la barra de direcciones. Se probó también el acceso directo por HTTP, y este fue redirigido automáticamente a la versión segura del sitio. De esta manera, tanto el acceso a la interfaz como la transmisión de credenciales, archivos y datos sensibles se realiza de forma cifrada, cumpliendo con los estándares actuales de protección de datos.

En conjunto, las pruebas realizadas demostraron que Super Ticket es una plataforma funcional y segura, capaz de operar sobre una infraestructura gratuita en AWS, proporcionando soporte a múltiples empresas con una única instancia. El rendimiento fue

satisfactorio, el modelo multiempresa funcionó correctamente y todas las medidas de seguridad aplicadas —como el uso de HTTPS, IAM, S3 privado y logs— se validaron con éxito.

## Capítulo 9: CONCLUSIONES Y POSIBLES

### MEJORAS

#### 9. 1. RESUMEN DE LOGROS

A lo largo del presente proyecto se ha logrado diseñar, adaptar y desplegar una plataforma web funcional de ticketing multiempresa, bautizada como Super Ticket, sobre infraestructura 100 % en la nube, utilizando exclusivamente recursos disponibles dentro del nivel gratuito (Free Tier) de Amazon Web Services (AWS).

El sistema se ha construido a partir de la base del software de código abierto Ticketly, al que se le ha incorporado soporte multiempresa mediante un rediseño de la base de datos, introduciendo el campo empresa\_id en las entidades principales. Esta adaptación ha permitido que múltiples organizaciones compartan una misma instancia de la plataforma, manteniendo el aislamiento de datos y una gestión segmentada.

La infraestructura se desplegó con tecnologías modernas como Docker, Amazon ECS, RDS para la base de datos MySQL, S3 para el almacenamiento de adjuntos, ACM para el cifrado HTTPS, y CloudWatch para el registro de logs, todo orquestado con Terraform como herramienta de infraestructura como código. Las pruebas han demostrado que el sistema funciona correctamente, respeta la segmentación entre empresas, y se mantiene operativo con buen rendimiento en una instancia EC2 t2.micro, confirmando su idoneidad como proyecto formativo y como prototipo técnico funcional.

## 9. 2. LIMITACIONES ACTUALES

A pesar del éxito técnico alcanzado, Super Ticket presenta actualmente varias limitaciones derivadas tanto de su diseño inicial como del entorno de ejecución utilizado:

- Escalabilidad limitada: el uso de una sola instancia EC2 con recursos mínimos restringe el número de usuarios concurrentes y la carga total que puede manejarse. Para entornos de producción real, sería necesario escalar horizontalmente los servicios, lo que ya no entra dentro del Free Tier.
- Interfaz básica: aunque funcional, la interfaz de usuario de Super Ticket es sencilla y poco atractiva visualmente. Carece de componentes modernos, como gráficos o dashboards en tiempo real.
- Ausencia de API externa: actualmente, la interacción con el sistema se realiza solo desde la interfaz web. No existe una API RESTful formalizada que permita la integración con otros sistemas externos o clientes móviles.
- Gestión básica de sesiones: la autenticación se realiza mediante sesiones PHP tradicionales, sin uso de tokens ni autenticación avanzada.
- Monitoreo limitado: aunque se han incluido logs básicos con CloudWatch, el sistema carece de dashboards de métricas o alertas automáticas en caso de caídas o fallos graves.

Estas limitaciones no impiden el uso del sistema como prueba de concepto o herramienta de aprendizaje, pero sí delimitan claramente su ámbito operativo actual.

## 9. 3. MEJORAS FUTURAS

De cara a una evolución del proyecto, se identifican varias mejoras prioritarias que permitirían transformar Super Ticket en una solución más robusta, moderna y escalable:

1. Autenticación mediante JWT (JSON Web Tokens): Implementar un sistema de autenticación basado en tokens firmados permitiría separar el frontend del backend, mejorar la seguridad y facilitar el desarrollo de clientes móviles o SPA (Single Page Applications).
2. Interfaz de usuario más moderna: Reescribir la interfaz utilizando frameworks como Vue.js, React o incluso Alpine.js, permitiría dotar al sistema de una apariencia más profesional, incluyendo interacción dinámica, notificaciones en tiempo real y gráficos interactivos.
3. API RESTful documentada: Exponer las funcionalidades del sistema a través de una API RESTful permitiría integraciones con otros sistemas, soporte para automatización, o creación de apps móviles. Esta API debería incluir autenticación con JWT y estar bien documentada con Swagger/OpenAPI.
4. Comparativa con DynamoDB: Como parte de una exploración más avanzada, se propone implementar una versión alternativa del backend utilizando Amazon DynamoDB como base de datos NoSQL, y comparar su rendimiento y escalabilidad frente a MySQL, especialmente en entornos con alta concurrencia.
5. Backups automáticos de RDS hacia S3: Aunque RDS incluye backups automáticos por defecto, sería deseable automatizar la exportación periódica de snapshots a buckets S3, facilitando la recuperación de desastres o la replicación entre regiones. Esta tarea puede integrarse con AWS Backup o mediante scripts Lambda programados.
6. Integración con herramientas de monitorización: La incorporación de servicios como Amazon CloudWatch Dashboards, Prometheus o Grafana permitiría supervisar métricas clave (uso de CPU, errores HTTP, tiempo medio de respuesta) y actuar rápidamente ante incidentes.

## 9. 4. REFLEXIÓN FINAL

Super Ticket ha demostrado ser una plataforma sólida y adaptable, capaz de operar con garantías sobre una arquitectura moderna basada en servicios gestionados. Su diseño multiempresa, la implementación sobre contenedores y la gestión de infraestructura con Terraform la convierten en un ejemplo claro de cómo construir soluciones web profesionales dentro de los límites del Free Tier de AWS.

Este proyecto no solo ha permitido comprender los fundamentos del desarrollo y despliegue en la nube, sino también poner en práctica principios clave como la separación de responsabilidades, el diseño modular y la seguridad en entornos productivos. Las mejoras propuestas abren la puerta a una evolución real del sistema hacia estándares de producción más exigentes, manteniendo como base su filosofía abierta y su enfoque didáctico.

### ○ BIBLIOGRAFIA

- <https://aws.amazon.com/free> –
- <https://docs.aws.amazon.com/>
  - [Amazon EC2](#)
  - [Amazon ECS](#)
  - [Amazon RDS \(MySQL\)](#)
  - [Amazon S3](#)
  - [AWS Certificate Manager \(ACM\)](#)
  - [AWS CloudWatch](#)
- <https://docs.aws.amazon.com/wellarchitected/>
- <https://owasp.org/>
- <https://docs.aws.amazon.com/IAM/>
- <https://developer.hashicorp.com/terraform>
- <https://registry.terraform.io>
- <https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html>
- <https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>
- <https://docs.aws.amazon.com/acm/latest/userguide/>
- <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

- <https://owasp.org/www-project-top-ten/>
- <https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html>
- <https://developer.hashicorp.com/terraform/docs>
- <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- <https://developer.hashicorp.com/terraform/learn>
- <https://docs.gitlab.com/ee/ci/>
- <https://docs.github.com/en/actions>
- <https://kubernetes.io/docs/home/>
- <https://helm.sh/docs/>
- <https://prometheus.io/docs/introduction/overview/>
- <https://grafana.com/docs/>
- <https://chatgpt.com/>
- <https://docs.pulumi.com/>
- <https://github.com/>

Comentado [jb1]: